

Cognitive Research: Principles and Implications

'Mathematics is holy mojo'

Why generative pre-trained transformer models are not stochastic parrots

--Manuscript Draft--

Manuscript Number:	CRPI-D-25-00223
Full Title:	'Mathematics is holy mojo' Why generative pre-trained transformer models are not stochastic parrots
Article Type:	Review article
Funding Information:	
Abstract:	The widespread critique that generative pre-trained transformer (GPT) models are "stochastic parrots" rests on a misunderstanding about how such models process and represent their training input. GPT models are not mere statistical mimics of linguistic form but are better understood as inductive inference engines that operate over rich, high-dimensional semantic spaces. Modern word embedding models, such as LSA and word2vec, reveal that the structure of meaning can be learned through statistical relationships between word contexts. GPT models extend this approach, learning to generate vectors not only from tracking word order but from dynamic attention-based transformations across contexts. These models navigate semantic space in a manner that parallels human inductive reasoning, often producing original and meaningful sentences that neither replicate nor resemble their training data. I show that such capabilities arise not despite, but because of, the non-semantic nature of their training objectives. Furthermore, I explain that GPT models do not represent words internally. They rather construct contextually appropriate vectors that usually do not correspond to any labeled word at all. Semantic generalization arises because the geometry of the model encodes a latent structure far richer than any discrete lexicon. The claim that GPT models merely repeat what they have seen collapses once we recognize that they were trained not to recognize words, but to predict contextually meaningful representations. These models are not stochastic parrots. They are semantic composers operating within a vast, unlabeled, meaning-saturated space.
Corresponding Author:	Chris Westbury University of Alberta CANADA
Corresponding Author E-Mail:	chrisw@ualberta.ca
Corresponding Author Secondary Information:	
Corresponding Author's Institution:	University of Alberta
Corresponding Author's Secondary Institution:	
First Author:	Chris Westbury
First Author Secondary Information:	
Order of Authors:	Chris Westbury
Order of Authors Secondary Information:	
Suggested Reviewers:	Randy Jamieson Randy.Jamieson@umanitoba.ca Expert in computational psycholinguistics Steven Piantadosi spiantado@gmail.com Expert in computational psycholinguistics; cited in submitted paper Emmanuel Keuleers E.A.Keuleers@tilburguniversity.edu Expert in computational psycholinguistics

Opposed Reviewers:	
Additional Information:	
Question	Response
Is this study a clinical trial? A clinical trial is defined by the World Health Organisation as 'any research study that prospectively assigns human participants or groups of humans to one or more health-related interventions to evaluate the effects on health outcomes'.	No
Are you submitting your manuscript to a Special Issue?	No

‘Mathematics is holy mojo’

Why generative pre-trained transformer models are not stochastic parrots

Chris Westbury

Department of Psychology, University of Alberta
P220 Biological Sciences Building
Edmonton, AB, Canada
T6G 2E9

Correspondence should be addressed to Chris Westbury at the above address or by e-mail:
chrisw@ualberta.ca.

Keywords: computational modeling, generative pre-trained transformer model, latent semantic analysis, semantics, stochastic parrot, word meaning, word2vec

Abstract word count: 235 words

Main text word count: 13338 words

References word count: 1370 words

Total word count: 16089 words

Abstract [235 words]

The widespread critique that generative pre-trained transformer (GPT) models are “stochastic parrots” rests on a misunderstanding about how such models process and represent their training input. GPT models are not mere statistical mimics of linguistic form but are better understood as inductive inference engines that operate over rich, high-dimensional semantic spaces. Modern word embedding models, such as LSA and word2vec, reveal that the structure of meaning can be learned through statistical relationships between word contexts. GPT models extend this approach, learning to generate vectors not only from tracking word order but from dynamic attention-based transformations across contexts. These models navigate semantic space in a manner that parallels human inductive reasoning, often producing original and meaningful sentences that neither replicate nor resemble their training data. I show that such capabilities arise not despite, but because of, the non-semantic nature of their training objectives. Furthermore, I explain that GPT models do not represent words internally. They rather construct contextually appropriate vectors that usually do not correspond to any labeled word at all. Semantic generalization arises because the geometry of the model encodes a latent structure far richer than any discrete lexicon. The claim that GPT models merely repeat what they have seen collapses once we recognize that they were trained not to recognize words, but to predict contextually meaningful representations. These models are not stochastic parrots. They are semantic composers operating within a vast, unlabeled, meaning-saturated space.

Significance Statement [197 words]

Generative pre-trained transformer (GPT) models such as ChatGPT have created the first widely accessible form of general artificial intelligence. In only a few years, they have launched a multi-billion-dollar industry and begun to reshape education, medicine, science, and work itself. Popular accounts often misrepresent how these systems function, repeating some form of the claim that they are merely “stochastic parrots”, elaborate echo machines that regurgitate text drawn from their training data. This misunderstanding has led many to dismiss their scientific and social significance. This paper explains why GPT models are not parrots. Drawing on the long scientific history of word-embedding research, it shows that these models do not memorize words but learn to represent *meanings* as patterns in a high-dimensional semantic space. Their power lies in discovering and manipulating relationships among meanings, allowing them to generalize to situations and sentences never encountered before. Properly understood, GPT models are not mimics but *semantic engines*: tools that translate between the continuous geometry of meaning and the discrete symbols of language. What we are witnessing is not a new trick of imitation, but the emergence of a new kind of cognitive instrument, one capable of something closely analogous to understanding.

The idea that categories can be recognizable without being strictly definable has been explored for decades, from Wittgenstein's (1953) notion of family resemblance to Polanyi's (1966) extensive discussion of tacit knowledge to Rosch's (1973; Rosch & Mervis, 1975) prototype theory and its extension into metaphor by Lakoff (1987). The human experience of recognizing a quality without being able to define it applies to the problem of recognizing what is meaningful. An oft-repeated claim about generative pre-trained transformer (GPT) models is that they do not operate with meaning at all but are rather "stochastic parrots"¹ (Bender, Gebru, McMillan-Major, & Shmitchell, 2021) or "money laundering for copyrighted data" (Willison, 2023) or "a blurry jpeg of the web" (Chiang, 2023) or "copy/paste on steroids" (Lee, 2023). In this paper I present evidence that and explain why these frequently expressed opinions are unfounded. Claims that GPT models can do no more than parrot their input text are based on at least two related errors, both of which may seem too obvious to doubt: that word meanings are simple computational units and that GPT models are designed to predict the next word.

In the next section I briefly summarize the history of thought on the nature of word meaning. In the third section I outline in more detail how word embedding models work and what they can do. In the fourth section I explain how GPT models work, with an emphasis on

¹ Bender et al.'s (2021) article did state the claim that a language model is a "system for haphazardly stitching together sequences of linguistic forms it has observed in its vast training data, according to probabilistic information about how they combine, but without any reference to meaning: a stochastic parrot" (p. 617). However, their paper focused primarily on the ethical, social, and environmental implications of large language models: their carbon footprint, biases, and the limited diversity of their training corpora. The present paper addresses only the narrow scientific claim that language models lack genuine semantic representation and merely recombine surface statistics.

their representation of semantic space. In the fifth section I present and discuss empirical evidence showing that GPT models and humans are largely in agreement about how meaningful sentences are. I will summarize the main points of the paper in a general discussion.

1. The Nature of Word Meaning

Despite its central place in psychology and the fact that all of us do know it when we see it, it is difficult to define what meaning is. Although it would be difficult to state unambiguously where to draw the line, somewhere between the detection of a single bit of information (an amoeba oozing along a temperature gradient or a tick detecting the presence of butyric acid) and the synthesis of thousands or millions of bits of information (a holy book or a hockey game), meaning arises. Even the seasoned pros at the Oxford English Dictionary threw up their hands, falling nearly into tautology by defining the word *meaning* broadly as the “significance, purpose, underlying truth, etc., of something” (Oxford English Dictionary, 2000).

This paper focuses on one form of meaning, lexical semantics. The term *semantics* was first coined by the philologist Michel Bréal in his (1883) article *Les lois intellectuelles du langage. Fragment de sémantique*. He suggested the term *semantics* (*sémantique*) for the study of linguistic meaning, from the Greek verb *σημαντικός* [*simantikós*], meaning ‘to signify’. Many theories have since been proposed to explain how words can signify something other than themselves.

Perhaps the oldest theory of lexical semantics, with roots going back to Socrates, is *the mentalist theory of meaning*, which argues that the meaning of a term is the mental image to which it refers. The mentalist theory has the major problem that it makes meaning subjective, although Socrates tried to get around that by improbably assuming the existence of universal forms in a perfect reality adjacent to the sad vale of tears we inhabit. If we forego that mystical

convenience, the meaning of the word *cat* might be entirely different for each of us. Depending how much weight is placed on appearance, the problem may be exacerbated for abstract terms, since we cannot always easily fix the meaning of abstract terms by comparing characteristics (Wittgenstein, 1953), as we might if a person were confusing, say, cats with skunks.

The mentalist theory of meaning has found contemporary extension in *conceptual semantics* (e.g., Wierzbicka, 1992; Jackendoff, 2002). The essential idea of conceptual semantics was stated by Chomsky (2000): “However surprising the conclusion may be that nature has provided us with an innate stock of concepts, and that the child’s task is to discover their labels, the empirical facts appear to leave open few other possibilities” (p. 64-66). This bold but dubious claim recalls Dennett’s (1991) definition of Philosopher’s Syndrome: “mistaking a failure of the imagination for an insight into necessity” (p. 401). Philosopher’s Syndrome impacts the entire history of academic thought about lexical semantics, including the way that most people think about GPT models. It is easy and common to mistakenly believe that our intuitions about a thing give us direct insight into the explanation of that thing. Conceptual semantics assumes that meaning is fixed by “a finite set of mental primitives and a finite set of principles of mental combination” (Jackendoff, 1990, p. 9), an assumption that makes semantic structures and principles analogous to the unconscious structures and principles posited by Chomskian syntacticians to underlie syntax. Conceptual semantics proposes that the semantic meaning of word is combined with a non-linguistic conceptual representation into a *lexical concept*, blurring the line between linguistic and non-linguistic meaning. Semantic structures are assumed to be composed of primitives (e.g., events, states, places, amounts, experiences, things, properties, emotions) that combine in rule-governed ways.

A looser version of the mentalist theory was outlined by several cognitive linguists beginning in the 1980s (e.g. Lakoff & Johnson, 1980, 1999, Lakoff, 1987) in highlighting the central role of metaphorical extension in lexical semantics. These cognitive linguists emphasized that many linguistic distinctions (e.g., front/back, inside/outside, over/under, up/down, part/whole) are rooted in sensory-motor and affective experience and that these embodied categories offer a fruitful path for metaphorical extension both in language and in our own private meaning-making.

The referential theory of meaning suggests that the meaning of a word is whatever it refers to, its denotation. The referential theory of meaning is problematic for several reasons. One is that it fails to capture the fact that words do not only have denotative meaning, but also connotative meaning, a set of affective, pragmatic, and conceptual associations. A second problem with referential theories of meaning was famously discussed by the philosopher Gottlob Frege (1892). He noted that the planet Venus has been referred to as both *the morning star* and *the evening star*. It seems clearly wrong to say that the tautological sentence *The morning star is the morning star* has the same meaning as sentence *The morning star is the evening star*, although the referential theory of meaning implies that the sentences have identical meaning. Frege proposed that we distinguish between a word's *reference* (denotative meaning) and its *sense* (our subjective understanding of the meaning). Senses are much more complex and numerous than denotative meanings.

Another theory of lexical semantics is *the behavioral theory of word meaning*. The idea is usually attributed to the linguist Leonard Bloomfield. Bloomfield (1933) defined the meaning of a word as “the situation in which the speaker utters it and the response which it calls forth in the hearer” (p. 139). This idea was picked up by Skinner in his (1957) book *Verbal Behavior* (see

Passos & Matos, 2007), which was famously denigrated in Chomsky's (1959) review of the book. Among a barrage of criticisms, Chomsky (1959) pointed out that "the record of inputs to the organism and the organism's present response [...] is nothing more than the definition of his problem" (p. 27). Bloomfield himself had previously recognized this issue, commenting about his own theory that: "In order to give a scientifically accurate definition of meaning for every form of a language, we should have to have a scientifically accurate knowledge of everything in the speaker's world." (Bloomfield, 1933, p. 139-140). This seems to be close to asserting that a behavioral theory of meaning places lexical semantics entirely beyond the reach of empirical study.

The use theory of meaning is related theory of meaning that posits that "the meaning of a word is its use in language" (Wittgenstein, 1953; see also de Saussure, 1916). The use theory addresses the problem of identifying the relevant inputs for a behavioral theory of word meaning, by narrowing the scope to accepted, publicly-observable uses of language. Under the use theory of word meaning, word meaning is not a function of the observable inputs of the speaker. It is rather a function of a speaker's interactive history with other language speakers who share what Wittgenstein referred to as the same 'form of life' (roughly, the same subculture). The meaning of a word is wholly contained in the historically-contextualized way that it is used.

The referential, behavioral, and use theories of word meaning are all broadly externalist. Externalists (e.g., Putnam, 1975; Burge, 1986) hold that word meaning depends largely on how words are anchored to the external world rather than on purely linguistic or inferential relations. In emphasizing word-to-world connections, they deemphasized the role of mental representations that had been central to earlier empiricist accounts of meaning. The position advanced in this paper can be viewed as statistical externalism, since transformer models (and

their distributional predecessors) internalize the statistical residue of word-to-world relations, compressing it into high-dimensional geometry.

The view I will outline below of GPT models as inductive inference engines navigating a structured semantic space intersects with several recent traditions in the philosophy of language and cognitive science, especially *conceptual-role semantics* and *inferentialist theories of meaning*. Conceptual-role semantics (Block, 1998; Harman, 1973) holds that a representation's meaning depends on its functional role in thought and inference, how it interacts with other representations, beliefs, and actions. It arose from efforts to explain meaning in terms of use and internal cognitive relations, drawing on Sellars' (1956) rejection of *the given* (knowledge grounded in raw, preconceptual experience) and on Wittgenstein's (1953) claim that meaning is grounded in linguistic practice.

Inferentialism (Brandom, 1994) develops this insight in a normative, social direction by locating meaning in the web of inferences that governs our shared commitments and entitlements, the pattern of reasons we use to justify what we say within discourse. Inferentialism thus systematizes Wittgenstein's more open-ended idea of "meaning as use." Conceptual-role semantics treats meaning as a property of mental architecture, whereas inferentialism treats it as a property of participation in shared reasoning practices. Both approaches replace earlier reference-based semantics with relational and functional accounts of meaning.

Conceptual-role semantics and inferentialist theories of meaning are consistent with connectionist and transformer models, which encode norms and patterns of reasoning to the extent that they can be induced from patterns of language use alone. Connectionist and transformer models also align with more recent proposals by Piantadosi and Hill (2022), who argue that "the search for meaning in learning models—or brains—should focus on

understanding the way that the systems' internal representational states relate *to each other*" (p. 3, emphasis in original). Transformer models give that proposal an explicit representational form. They learn not isolated lexical items but the inferential roles words play across contexts, thereby operationalizing use-theoretic semantics in a high-dimensional geometry.

The same geometric structure that allows GPT models to generalize meaningfully from context also links them to contemporary cognitive neuroscience. A recent (not yet peer-reviewed) study comparing neural activation in humans and large language models found convergent representational topographies (Katlowitz et al., 2025), suggesting that both systems may rely on analogous high-dimensional inductive compression. This finding is consistent with earlier work that has found more abstract similarities between measures derived from GPT models and measures of brain activation (Schrimpf et al., 2021; Goldstein et al., 2022; Heilbron et al., 2022; Goldstein, et al., 2023; Kauf et al., 2024) The capacity of GPT models to predict human judgments of meaningfulness, as demonstrated below, is therefore consistent with emerging evidence that linguistic brains and language models share a common informational architecture rather than a merely superficial resemblance.

2. Word-Embedding Models

In all the theories of lexical semantics considered so far, word meaning is abstract and difficult to assess empirically. However, the use theory of meaning is closely related to a modern theory of lexical semantics that makes the use theory concrete and amenable to empirical enquiry: *the distributional theory of meaning*. Distributional theories narrow the wide scope of use theories in a way that makes them directly amenable to computational modeling and testing.

The precursor to the defining idea of distributional theories of word meaning was first specifically stated by Joos (1950): “[...] the ‘meaning’ of a morpheme (or of a combination thereof, of any complexity up to a complete utterance, which might even be a whole book or poem) is by definition the set of conditional probabilities of its occurrence in context with all other morphemes—of course, without enquiry into the outside, practical, or sociologist’s meaning of any of them” (p. 708). Harris (1954) made a very similar point. Distributional theories of meaning have become popular recently, in large part precisely because they are defined specifically enough to be modeled computationally. The first computational models based on word distribution began by simply counting how often any two words co-occurred within the same document (Latent Semantic Analysis [LSA]; Landauer & Dumais, 1997) or within the same small moving window of text (Hyper-linguistic Analog to Language [HAL]; Lund & Burgess, 1996).

2.1 Latent Semantic Analysis [LSA]

Although the models are conceptually nearly identical, I will focus on explaining how LSA works, for two reasons. One is because LSA is deals with language in a slightly more concrete way than HAL, since it focuses on whole documents rather than an arbitrary computationally-defined context. The more important reason is that Landauer & Dumais’ (1997) paper was explicitly focused on explaining how word-embedding models are related to *induction*, the ability to reason from observed particulars (say, all the cats you have ever met) to induced generalities (your concept of a cat). Understanding how induction is related to word-embedding is important to understanding why GPT models are not simply stochastic parrots.

The LSA model is simple. The key insight is an idea we have already discussed: that words with similar meanings tend to occur in similar contexts. Landauer and Dumais’s

contribution was to point out that this observation suggested a formal method for estimating how similar two different word meanings are. The first step of their method is to gather many documents. The second step is to how often any two words that co-occur in each document, keeping a running tally.

Consider three very short documents, A, B, and C, each of which contains five words:

“To be is to do”—Martin Heidegger.

“To do is to be”—Jean-Paul Sartre.

“Do be do be do”—Frank Sinatra.

If we tally up the occurrence of the words in each of these documents (ignoring the proper names, just to make this toy example more transparent), we end up with Table 1. The table shows four dimensions [rows] for describing our three documents, corresponding to the four words that appear in at least one of the documents. We can see that our tiny documents A and B contain the same words and that those words occur the same number of times. We can also see that documents A and B are quite different from document C. There are several ways to demonstrate this mathematically because there are several ways to compute the similarity of vectors (the rows in Table 1).

A common way of comparing vectors is to use *cosine similarity*, so-called because it measures the cosine of the angle between two vectors. This is defined as the dot product of the vectors divided by the product of their lengths:

$$\frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

The cosine similarity of the vectors for document A and B is 1, signalling that they are identical (again, ignoring word order for now). Comparing either document A or B to document

C, the cosine similarity is 0.52. These simple distance measures tell us what is obvious by inspection: that document C is more different from document A or B than document A is from document B.

LSA is not simply summing up how often two words co-occur in the same document, which would provide us with one measure for each pair of words. That direct co-occurrence is known as *first-order co-occurrence*. We can use first-order co-occurrence to get a little bit of information about word meanings. For example, if we found the word *dog* occurred in the same document much more often with the word *cat* than with the word *ambiguity*, we might induce that cats and dogs are more similar than dogs and ambiguity. However, LSA uses the simplicity of counting first-order co-occurrence only as a step towards computing the similarity between *the contexts* in which two words occur, as represented by their vectors. We call this *second-order co-occurrence* because is it not about whether any two words ever occur close together (first-order co-occurrence), but rather about whether any two words ever occur in similar contexts (second-order co-occurrence).

The context similarity measured by second-order co-occurrence underlies our understanding of how LSA uses induction. Words can have similar (theoretically, even identical) contexts without ever appearing in the same document. Similarity of context means that the words co-occurred with many of the same words, rather than that they occurred with each other. LSA can therefore induce that two words are similar even if those two words never appear in the same document.

The second reason that the vector's encoding of second-order co-occurrence is important is that it underlies the huge success of GPT models. People sometimes seem to treat a word's vector as if it were just an arbitrary symbol of that word, as if we had proclaimed: *Let's call the*

word *dog* 'x'. When I say *I love my x*, you know that I love my dog, because you can understand that x is just an arbitrary symbol that we agreed would stand in for the word *dog*. We didn't need to call it x. Since variable names are arbitrary, we could have called it anything: *foobar*, *Agamemnon*, *Petunia_Pieface*, and so on.

However, a word's vector is not an arbitrary variable name but is rather a compressed encoding of that word's context. We can't simply switch the labels of the vectors for the words *cat* and *dog*, which we would do if the vectors were only arbitrary symbols. If we switched the labels for the vectors from a word embedding model, then our word embedding model would think that dogs were cats, and vice versa, because those vectors directly encode information about what it means to be a dog or a cat.

Being able to compute the similarity of contexts allows for the drawing of analogies between two things that have never been associated in experience using the classical tools of associationism: contiguity in time or space, or graded resemblance. This drawing of analogies from second-order co-occurrence is induction, since we are abstracting from particular observations (how often words occur together) to make conclusions about general truths (how similar words are in meaning).

First-order associationism has been accorded a highly misleading weight in psychology (see Rescorla, 1988). However, the 18th century Scottish philosopher David Hume did recognize that second-order associationism might be important. He noted that two things could become associated even if they never occurred together and did not resemble each other, if they were both associated with a third object. In his (1739) *Treatise of Human Nature*, the skeptical philosopher wrote: "That we may understand the full extent of these [associationist] relations, we must consider, [sic] that two objects are connected together in the imagination, not only when the

one is immediately resembling, contiguous to, or the cause of another, but also when there is interposed betwixt them a third object, which bears to them any of these relations” (p. 55). The last phrase is essentially a description of a simple form of second-order co-occurrence. Two things can become related to each other because something else is interposed betwixt them.

Word-embedding models like LSA and HAL do something extraordinary: they bootstrap meaning, the apparently magical property that defies simple definition, from the purely mechanical process of counting how often words co-occur within defined sets of words. I speak no Mandarin but in theory I can construct a co-occurrence matrix in Mandarin by hand, because the process of construction only requires me to recognize and count Mandarin characters. I don’t need to know what they mean.

It is illuminating to consider the situation of a non-Mandarin speaker with a Mandarin word-embedding matrix, in relation to John Searle’s famous (1982) Chinese Room Puzzle. Searle asked us to imagine a non-Mandarin speaker manipulating cards in some rule-based way to answer Mandarin questions. Searle points out that neither the card-manipulating human being nor the entire system speaks Mandarin, drawing a strong line between language users and non-users. Word embedding models clarify that linguistic competence resists such a simplistic binary categorization. Having access to a Mandarin word embedding matrix of course does not make me a Mandarin speaker. However, it does enable me to induce some knowledge, such as that 猫 [cat] is used in similar ways to how 狗 [dog] is used. I can do much more language-like work with a Mandarin word-embedding matrix. For example, I could use the matrix to make cluster diagrams to discern semantic relationships based on the similarity of the vectors of large groups of words. If I knew just the Mandarin word for *happy*, I could bootstrap that tiny bit of knowledge to make a simple model of emotional valence (see Hollis, Westbury, & Lefsrud,

2017). If I had some ratings of single word humour in Mandarin, I could make progress towards modeling those ratings (see Westbury & Hollis, 2019).

None of this is close to linguistic fluency, but it is not total linguistic ignorance either. Word-embedding models make clear that language use exists on a continuum, from passive geometric navigation or semantic structure identification (such as I can do when I have a Mandarin word-embedding matrix) to the full generative participation that a Mandarin speaker is able to do. As Wittgenstein noted in his (1953) *Philosophical Investigations*: “Every sign by *itself* seems dead. *What* gives it life? – In use it is *alive*.” (p. 128; italics as in the original).

The question we need to be considering when we ask to what degree a language model is ‘a true language user’ is not whether a language model really understands language in some metaphysical sense. The question is rather: To what degree can the model participate in the *practice* of language? By any behavioral, inferential, or functional standard, a person or a machine that has access to a word-embedding matrix of a language they do not speak can participate more in the practice of that language than that same person or machine could without such a matrix.

Landauer and Dumais (1997) proposed their model to address a language-related induction mystery, alluded to in the title of their paper, *A Solution to Plato’s Problem*. The “problem” is so called because its earliest statement appears in Plato’s *Meno*: we know things that we do not know that we know. In the dialogue, Socrates leads his companion Meno and a slave boy through a geometric proof that the boy appears able to solve despite having no education. Plato used this episode to argue that knowledge is *anamnesis*, the recollection of innate forms.

Following Landauer and Dumais, we can interpret this not as a literal doctrine of innate knowledge but as a metaphor for latent structure recovery which can explain how new understanding can arise from implicit relations already present in experience. This is the sense in which Plato's Problem anticipates the logic of distributional semantics and the operation of GPT models. Both demonstrate that generalization can emerge from the relational geometry of prior experience rather than from explicit instruction.

Landauer and Dumais (1997) illustrated this by noting that a typical American seventh grader learns 10–15 new word meanings daily, despite direct instruction accounting for almost none of them. Their LSA model simulated this apparent miracle of learning “unseen words” through induction from contextual similarity. The model's representation of a word changes each time it encounters a semantically related word, thereby updating its estimate of meaning even for words never directly observed.

LSA's solution to Plato's problem has a radical implication: that the “relation between any two representations depends not only on direct experience with them, but with everything else ever experienced” (Landauer & Dumais, 1997, p. 217). In a word-embedding model, this conclusion follows directly from the fact that the similarity between the vectors of two words will change every time we encounter *either* word. Since the similarity between any two words in a vector model is well-defined, a change in the vector representation of one word must change that vector's similarity relationship to every other vector in the matrix. In a high dimensional space, every time we learn something about one word, we learn a little more about *every* word. Words are dynamic representations that are continually being re-activated and re-shaped through language use. This has implications for the most famous expression of Plato's problem in modern times, Chomsky's (1980, 1986) claim about ‘the poverty of the stimulus’. If we think of

language as a resonant structure in which information about *any* word provides us with a little more information about every word, then a child’s language experience is much richer than Chomsky was ever able to imagine it to be.

If we tried to make the ‘stochastic parrot’ criticism of LSA, it would be “All that LSA can do is tell you what it was trained on: how often words co-occur in the same documents”. This is clearly untrue. An LSA matrix was never intended to be used to estimate how often word pairs appeared in the same document. The power of the model arises from the fact that the vectors represent an aggregate description of the context of *every* word. Counting how often words occur in the same document was just a way to get those aggregate descriptions.

It is not the only way to get them. The development of HAL and LSA paved the way for many other word-embedding models to be implemented (e.g. BEAGLE, Jones, Kintsch, & Mewhort, 2006; word2vec, Mikolov, Chen, Corrado, & Dean., 2013; GloVe, Pennington, Socher, & Manning, 2014; FastText, Bojanowski, Grave, Joulin, & Mikolov, 2017; BERT, Devlin, Chang, Lee, & Toutanova, 2019). Although these differ in various ways from the early models and from each other, they are all conceptual cousins of those models. The main advance in the more recent models is how the context vectors are derived.

2.2 Word2vec

Google’s word2vec model (Mikolov, Chen, Corrado, & Dean, 2013) is the most widely used of these models. It trained a simple neural network to predict first-order co-occurrence, i.e. to predict a target word given that word’s context². The input to the neural network during training is a just a vector the length of the dictionary with context words marked by giving them

² Or the opposite: a neural network that tries to a target word’s context given that target word. For our purposes, we can ignore the differences between these two approaches. Both produce vectors that encode second-order co-occurrence.

a 1 and every other word a 0 (multi-hot encoding). The output layer is a vector the length of the dictionary with the estimated probability of each word being the target word. There is one hidden layer (usually but not necessarily of length 300) between the input and the output, which is used as the vector for that target word. As a hidden layer, it contains weights that mediate between the input layer and the output layer. If the model's output is a bad guess during training (if it does not give a high probability to the target word) then back propagation is used to adjust the weights in the hidden layer to improve the prediction. Though computed in a very different way, a word's hidden layer is conceptually much like the vectors from an LSA model, an abstract representation of that word's average context.

To get a feeling for what the word2vec algorithm is doing, imagine that I ask you to guess what word is missing in the phrase *I love ____ pet dog*. Of course, there is no way to be sure what the missing word is. Many words and phrases might work there: *your, every, roasting*, and so on. Probably most of us would guess that the missing word is *my*. At the same time, there are many words that we would never guess because they could never occur there: *aardvark, Superman, went*, and so on. Word2vec learns during training how to treat words as discrete cues for other words. It can do this because words *are* reliable cues for other words, due to the fact the occurrences of different words are correlated.

Just as counting how often symbols occurred in the same document for LSA does not require that the counter understand the *meaning* of the symbols, so back-propagating information about every symbol's context in a word2vec model (though it would be tedious for a human to perform by hand) does not require understanding of the symbols' meanings. Like LSA and HAL, neural-network-based models such as word2vec bootstrap meaning-rich context vectors from a meaningless, mechanical pattern-matching exercise. This is important to stress because it

demonstrates again that what a word-embedding model is trained to do and what it can do after it has been trained do not have to be the same thing.

3. Generative pre-trained transformer models

The exact same principle holds for GPT models, to which we can now turn our attention. As we shall see, the fact that GPT models are trained on predicting the next word does not mean that all that they can do after training is to predict the next word. The transformer architecture was first introduced in a short paper out of Google that was published in 2017 (Vaswani, Shazeer, Parmar, et al., 2017), which proposed the architecture for translating between two languages. OpenAI later adapted the transformer architecture for autoregressive, generatively pre-trained models, introducing the term Generative Pretrained Transformer when it released GPT-1 in 2018 (Radford et al., 2018).

A GPT model is a neural network composed of multiple computational elements called *heads* that are each composed of three pre-trained parameter matrices: a query matrix, a key matrix, and a value matrix. Each matrix is a linear transformer that projects an input matrix down to a lower dimensionality, for reasons I will explain below. As a soundbite mnemonic (re-purposing Crain & Steedman's (1985) comment about syntax and semantics), we can say that the query matrix *proposes*, the key matrix *disposes*, and the value matrix *composes*. This mnemonic is purely metaphorical, of course. The query, key, and value matrices contain no explicit propositional content. They are just linear projections that allow the model to compute weighted similarities between contextual embeddings. The "propose/dispose/compose" phrasing is intended here to capture their respective mathematical roles in allocating and transforming the relevance of words in the context. I will begin by describing each of the three matrices in broad

functional terms and then discuss exactly what each matrix takes as input and produces as output within a GPT model.

3.1 The Query Matrix Proposes

We can think of a query matrix as sending propositions to its head's key matrix: *Here's what I propose we should pay attention to. Do you agree?* Both the query and key matrices consist of vectors that are associated with words in the current input context (in chat: the current conversation). When the models are in productive operation, the query vectors for all words in the current input context are compared with the key vectors for all words in the current input context, by calculating their dot product similarity. This comparison, the query, functions like a search for relevant information about words in the context, helping the model determine which tokens in are most relevant in the current context. Of course, the model isn't explicitly searching for or formulating questions about aspects of the words. Rather, the similarity between the query and key vectors is used as a weight on 'attention'³. The query vector gets 'an answer' to its query about what is relevant by computing how well its vectors align with the analogous key vectors, allowing the model to identify and weight contextually relevant words.

3.2 The Key Matrix Disposes

I say that the key matrix disposes because it 'answers' every query posed by a query vector. The match between a query vector and its analogous key vector results in attentional/relevance weight on the words that is proportional to the dot product between those vectors.

³ It would have been more accurate to call it *relevance*, because the word *attention* has some misleading connotations in this context. Most notably, we sometimes think of attention is being focused like a spotlight on one single element of the environment, whereas we normally think of *relevance* as being estimated for multiple elements in the environment.

You might wonder: *What makes the vectors from the key matrix so smart? How do they know so much about what's relevant?* The answer to this question is simple and takes us to the heart of what each head is computing. A key vector can respond appropriately to a query vector ('disposing' of its question) simply because the key matrix is trained to be more like the query vector to the extent that such similarity helps achieve the goal of selecting the next word in the input during training. A key matrix is 'so smart' at this task for the same reason that word2vec vectors are so smart at representing second-order co-occurrence of a word: because that is what they were trained to do. Note that in using these heads to rate relevance, we are bootstrapping something new from the training task. Rating contextual relevance is not the same as predicting the next word.

3.3 The Value Matrix Composes

The relevance-estimating match between vectors from the query and key matrices in each head is just a single value for each word in the current (translation or chat) context. To use those values, they need to be passed to the next component of the model. The value matrix performs this task. The vectors in the value matrix perform this task, carry the contextual information forward. The attention/relevance weights derived from the match between query and key vectors are applied to the value matrix to produce context-adjusted word-embedding vectors. These vectors contain the information that transforms the initial static word embedding vectors into context-specific vectors that dynamically capture the changing semantics of the context as each new output word is selected. While this description is metaphorical, the claim that transformer attention yields semantically structured contextual representations is empirically testable. Analyses of open-source models (e.g., Chronis & Erk, 2020; Misra et al., 2023) have shown that contextual embeddings capture graded semantic similarity and typicality, aligning closely with

human judgments. Such work supports the idea that attention mechanisms do more than allocate weights among tokens. They construct high-dimensional semantic representations that function as the model's working meanings.

Many interpretations of transformer function have been proposed, notably including Shalizi's (2023) mathematically formal explanation of attention (a term he despises) as learned adaptive smoothing over context. Shalizi came up with this idea independently but admits that priority is due to Tsai et al, 2019. Shalizi notes that the 'attentional weighting' operation undertaken by a GPT model is mathematically close to the kernel smoothing operation invented by Nadaraya (1964) and Watson (1964) many years ago. Kernel smoothing estimates a smooth curve in noisy, nonlinear data by locally averaging nearby observations, an idea that was later used in generalized additive models (Hastie & Tibshirani, 1987). In transformers, the same logic applies. The dot-product similarity between a query vector and a set of key vectors acts as a learned kernel, since those similarity measures quantify how relevant each key is to the query. A softmax function converts those similarities into normalized weights that sum to one, so the model is effectively computing a weighted average of the corresponding *value* (query/key similarity) vectors. Each token's new representation is a smooth interpolation among its contextual neighbors in word embedding space, with closer (more similar) contexts exerting greater influence. Compositional meaning then arises through local smoothing in a learned semantic geometry rather than through discrete symbolic substitution. This account complements and clarifies, rather than contradicts, the more accessible but less precise geometric interpretation I have offered of attention as context-sensitive semantic weighting.

3.4 Putting Our Heads Together

A head in a GPT model handles a lot of computation. The power of GPT models derives in large part from the fact that they use many heads. For example, consider GPT-3, a now-obsolete early model which I will discuss here because its structure is publicly known, which is not the case for more recent models. GPT-3 consists of 96 independent columns of heads. Each column in GPT-3 is composed of 96 heads. The model therefore has $96 \times 96 = 9216$ heads.

Figure 1 shows a simplified diagram of the model.

Why are there so many independent columns of heads? The paper introducing GPTs was titled *Attention is all you need* because it was premised on the idea that it was possible to trade time for attention by processing multiple (in this case, 96) streams of attentional/relevance analysis in parallel. In GPT-3, each of the 96 columns receives the same input, which is an input matrix of length 12,288 (the length of vectors in the GPT models word-embedding matrix) and of height equal to the context size. This is shown in the box at the top of Figure 1. All the words in the present context are included because the attentional/relevance computation makes pairwise comparisons between them all. In Figure 1, the model is in the process of finishing the phrase *A cat sat on the*. In the system state that is shown in the figure, it has processed the first four words, which are therefore marked as ‘context-adjusted’, since each pass through the GPT adjusts all the vectors in the current context to fit that context. On the previous step, the model chose the word *the* as the next word to output, so the raw word-embedding vector for that word (marked ‘non-adjusted’ at the top of Figure 1) has been appended to the context matrix.

The initial head reduces the input vectors in the context matrix, which are of length 12,288, to vectors of length 128 in the query, key, and value matrices. The astute reader may have deduced the nature of the relationship between the length of the word-embedding vector

(12,288) and the length of vectors used in the heads (128), because $12,288/128 = 96$, the number of columns in GPT-3. The word embedding vectors in a GPT-3 model are constructed by concatenating the final output vectors from the 96 columns of heads. In this way they are dissimilar from the vectors from word-embedding models discussed above. GPT-3 model vectors can be thought of as a record of 96 different ‘takes’ on word’s context/meaning.

This slightly over-simplifies matters because the heads are not the only components of a GPT model that process the vectors in their current context. After the initial head has processed all the vectors, the outputs from all 96 columns are concatenated, creating new word vectors of length $96 \times 128 = 12,288$. These output vectors are passed through a fully-connected feedforward network that ‘stretches’ them to four times their length and then compresses them back again to their original length of 12,288. This is done individually for each token, in contrast to the heads, which are necessarily processing the entire context because the attention/relevance calculation requires that each word be considered against every other word in the context. The token-focused step using the feedforward network is one reason that it was an over-simplification to write above that ‘GPT-3 model vectors can be thought of as a record of 96 different takes on word’s context/meaning’. The feedforward network introduces a non-linear element to the model’s production process, allowing the model to incorporate richer relationships between features relevant to individual context-adjusted tokens. The model is not, for example, simply ‘thinking more deeply’ about, say, a cat. It is thinking more deeply about a cat within the current context, which in the case shown in Figure 1 is about a cat that is sitting on...something unknown.

The processes of i.) computing attention/relevance for every word in the context and ii.) adjusting individual token vectors via the feedforward network are repeated for every head in each column = 96 times for ChatGPT-3. Two important questions are: Why are there so many

columns and what stops them all from picking up on the same main themes in the input text? (i.e., what guarantees that there is additional information in the $N+1^{\text{st}}$ column after the preceding N columns has already processed the same input?). There are three main reasons that the 96 different takes on the input are likely to be different takes.

Two reasons are that the heads are initialized randomly prior to training the model and that every head is trained independently from the other heads. However, these two reasons are weak. Different heads might converge on the same main themes in any given input text despite starting from different initial conditions and being trained independently. The third reason that the 96 columns are independent is that during training, any redundancy in the columns would result in sub-optimal performance at the non-semantic task used for training (predicting the next word), compared to using non-redundant columns. Redundant columns account for shared variance rather than picking up on new variance. The training process thus necessarily rewards diversity in the columnar computations. This is analogous to the reason that multiple regression models reward diversity in predictors. A predictor cannot be useful if the variance it explains in the dependent variable has already been explained by another predictor that has already entered the model. The GPT model is trying to construct *orthogonal* vectors that are relevant to its non-semantic training task, predicting the next word.

The last (simplified and trivial) step in a GPT model is shown the bottom of Figure 1. After the phrase *a cat sat on the* has been processed, it is only the context-adjusted vector of the final word *the* that is used to select the vector for the next word to output. This is not the original word-embedding vector for the word *the*. It is that vector after it has been adjusted to the entire context of the conversation by the $96 \times 96 = 9216$ heads, and the 96 feed-forward networks that

connect those heads. The next word is chosen by choosing a similar (not necessarily always *the* most similar) vector to that context-adjusted vector.

In the example in Figure 1, that next word might be *mat*. For the model to select the vector whose label is *mat*, the vector that was originally trained to represent the context of the word *the* must be adjusted by the model so that it is similar to the vector for the word *mat*. A GPT model chains words together by adjusting each final word vector (the vector for the word *the*) to be similar to the vector of a plausible following word (the vector for the word *mat*). This chameleon-like behavior of a word vector might make us all question what lexical semantics really is. In a GPT model during processing, a word does not have a single representation, although each word does have a generic starting representation stored in the word-embedding matrix. A word comes alive in use because every word has a nearly-infinite number of representations that reflect its context of use. If the current phrase was *a cat fell off the*, then the adjusted vector representation of the final word *the* would be less like the vector for the word *mat* and more like the vector for the word *desk* or (if it were an alcoholic cat) *wagon*. An example of this happening is shown in Figure 2.

Being trained to predict *vectors* may seem like a non-difference from being trained to predict a vector *label* (i.e. a word). However, the difference between predicting a word's vector and predicting a vector's label (a word) turns out to be very important. As discussed above, a word vector is not just an arbitrary symbol of its label but is instead a systematic high-dimensional representation of that label's context. A word embedding vector is more like a photograph or a sketch than it is like a variable name. A word vector does not merely *refer to* or *stand in for* its referent but rather *describes* that referent. GPT models are not trained to predict a vector label. They are trained to estimate the complex semantic environment in which that vector

label occurs.

4. What's in a vector?

We have so far reviewed some historical conceptual models of semantics and laid out the general principles by which a second-order vector can be created in different semantic models. In this section we will look more deeply into what information is contained in those second-order vectors.

The relationship that a word embedding vector has to its label's referent or connotations is shown in cartoon form in Figure 3, an imaginary representation of the vector for the word *dog*. The illustrative features are drawn from the closest vector neighbours to the word2vec vector for the word *dog* (using the Google News Corpus matrix available from <https://code.google.com/archive/p/word2vec/>, which I will use for all vector computations discussed in this paper). These neighbours include, for example, the words *cat*, *canine*, *puppy*, *doggy*, and *veterinarian*. Note that these are not just near-synonyms for the word *dog*. They represent a range of semantically-related words.

There are three points worth focusing on in Figure 3.

4.1 Vector Values Are Not Semantic Dimensions

One is that each neighbour is (of course) not represented by a single vector value but rather by a set of vector values. When we think about the semantic dimensionality of a word-embedding model, we must think in terms of *combinations* of vector values. A word vector is like a fingerprint. It has an identity not because each component has a named meaning, but because the overall pattern is unique, stable, and richly informative. No one bit tells you it's a dog; the whole constellation does. We cannot say that vector value 17 means 'animal-ness' and vector value 103 means 'attracts fleas'. But the combination of all 300 vector values lets us

reliably find other words that share those features, as well as features that have not been named by humans.

It is impossible to say how many vector values are implicated in any semantic dimension, because vector lengths in word-embedding modes are of arbitrary length and because different semantic categories may need a different number of vector values to ‘pin down’ their category, depending on how complex, subtle, or common it is. However, it is possible to make a principled estimate of the total number of semantic dimensions.

The upper bound on that number is mathematically precise because the number of combinations of N things taken M at a time is maximized when $M = N/2$. 12,288 things taken 6444 at a time gives an upper bound of 8.2×10^{3696} semantic dimensions that could be encoded with a ChatGPT-3 vector. This number, vast beyond human comprehension, can be put into perspective by noting that it is thousands of orders of magnitude larger than Eddington’s (1939) estimated number of atoms in the universe (10^{80}). If each of our possible semantic dimensions were represented by a single atom, we would need kajillions of universes to store them all.

What about the lower bound? The smallest possible number would be 12,288 taken one at a time, which would give us just 12,288 semantic dimensions. This is absurdly low and based on the absurd assumption that every vector value encodes exactly one semantic dimension. If there were ten vector values per semantic dimension (surely also far too low) then we would have 2.05×10^{34} semantic dimensions. If there were one hundred vector values per semantic dimension (almost certainly still too low) then we would have a semantic dimensionality of 3.88×10^{250} , nearly two hundred orders of magnitude larger than the estimated number of atoms in the universe. We could keep doing the math (the numbers climb rapidly), but the point is clear. The number of semantic dimensions encoded in GPT model vectors is somewhere between many

times more than the number of atoms in the universe and many, many times more than the number of atoms in the universe.

In the end, it probably makes no sense ask for a word-embedding model's 'true semantic dimensionality', since not all distinctions relevant to a GPT model's computation must necessarily correspond to a human-nameable category. We impose categories on GPT vectors, but the model does not need to break a vector down into what we decide are its component categories. Templeton et al. (2024) used computational methods to extract what they called 'monosemantic features' from the GPT model Claude 3 Sonnet. What they mean by 'monosemantic' is what most people mean by that term: 'human-nameable'. These features were not extracted from the output vectors but from examining the neural network itself. Examples of features they found included: Rwanda, the Golden Gate Bridge, transit infrastructure, Rosalind Franklin, sadness, the desire to be alone, sycophantic praise, scam e-mail, the brain sciences, immunocompromise, the non-word 'rihgt' in Python code only, transit infrastructure, and a feature that signals that addition is happening in Python. The number of such features was reported only as 'millions', which is surely a large underestimate that reflects that humans have attached names to only a tiny subset of all possible semantic features.

Far from avoiding vectors that don't have a human label attached, GPT models spend most of their time using unlabeled vectors for a reason we have already seen: because GPT models are specifically intended to modify the vectors that are given to them as input based on local context. When a GPT model produces a vector with a label that can complete the phrase *a cat sat on the*, it does so by modifying the vector for the final word it has produced so far, in this case the word *the*. The final vector is more like the vector for the word *mat* than it is like the vector for (say) the word *gorilla*. However, it is not the vector for the word *mat*. It is a mattish-

vector that has a cattish, sitting-on-like aspect to it. If we had to name its location in semantic space, it might be a mouthful like ‘things that afford sitting to a cat’.

Because they are built to modify word-embedding vectors to reflect context, GPT models rely almost entirely on vectors for unnamed locations in semantic space. Rather than thinking of GPT models as word-manipulating machines, it is better to think of them as semantic construction machines. The models were trained to predict the next vector that happens to have a label, but when they are computing what to produce as output, they are navigating through a very high dimensional space without any concern for whether their goal vector happens to have a label in the word-embedding matrix, which the main computational structure of the GPT model ‘cannot see’. When a GPT model chooses the word *mat* to complete the phrase *a cat sat on the*, it cannot do so by searching for the vector whose label is *mat* because a GPT model has no mechanism for searching for a vector with a given label. The model is instead constructing its own very specific cattish, sitting-affording vector. It only looks for the vector label of the labelled vector that happens to fall closest to the spot specified by that constructed cattish, sitting-affording vector when it needs to produce a vector label for a human.

In this sense GPT models are never ‘producing the next word’. They are building and refining semantic structures, most importantly the context-adjusted vector of the last word produced. Only after they have finished constructing the context-adjusted vector of the last word produced can a GPT look for the best human label for that structure. Choosing the next word is more like translation than it is like look-up, because the model is translating from its own internal representational space (which doesn’t know what a word is) to the most similar human-labeled vector. GPT models don’t use the labels that humans have attached to a vanishingly

small proportion of all possible vectors, which would be like tying one hand behind their back during a fight. They are not constrained to working only with those very rare vectors.

4.2 Meaning Is Compositional

This discussion is closely related to the second point that is apparent from Figure 3, which is that we can think of vectors with labels (i.e. word vectors) as the centroid of a set of other points that have labels, i.e. word meanings are compositional. The vector for (for example) the word *cat* can be defined as that point that lies at the centroid of the vectors for the set of all cattish things.

To illustrate how meaning is compositional in word embeddings, I implemented a greedy reconstruction algorithm that approximates the word2vec vector of a target word by summing a series of other word vectors. The Python code is available at <https://osf.io/6khqm/>. At each step, the algorithm selects a word from a randomly sampled subset of the vocabulary in Google's New Corpus word2vec matrix whose addition most increases the cosine similarity between the running sum and the target vector, ensuring monotonic convergence toward the target in semantic space. I settled on a tiny subset of 100 words to make the examples interestingly weird. With such a small sample, the meanings of the sampled words often bear little resemblance of the meaning of the target word. Nevertheless, their cumulative contribution reveals how even distantly related vectors can combine to reconstruct very specific meanings.

Figure 4 shows the convergence towards the vector for the word *cat* in the first run. In this case, the vector most similar to the vector for the word *cat* in the first 100 random words had the label *ringtail*. Although the cosine similarity between the two words is minimal, we can easily agree that ringtails (a kind of raccoon) and cats are similar in some ways: furry small mammals, at least. The second word (*yippie*) hardly improves the fit at all, but the next three

(*swan*, *beagle*, *puppy*) push the cosine similarity of the constructed vector with the vector for the word *cat* to 0.74, which is the same (to two decimal places) as the cosine similarity between the word2vec vectors for the word *cat* and the word *dog*. Many words with a very clear semantic relationship have vectors lower cosine similarities than that. For example, the cosine similarity between the word2vec vectors for the words *cat* and *kitten* is 0.59.

Of course there are many (approximately: infinitely many) other ways of constructing a vector similar to the vector of the word *cat* from other vectors that have a label, because we have many labeled vectors. Table 2 shows four other runs of our algorithm that end up constructing a vector with a cosine similarity to the vector with the label *cat* of at least 0.71. They are all very different, with little vocabulary overlap, but the way they circle around the idea of a cat is generally clear. If we increase the size of the random sample, the convergence is of course faster, since there is a higher chance of finding a word more closely synonymous to the target word. When we set the sample size to 5000, the first run the model converged on a vector that had a cosine similarity to the vector of the word *cat* of 0.80 using just the vectors from just five words: *doggie* (already quite similar to *cat*, with a cosine similarity to the word's vector of 0.49), *kitten*, *tomcats*, *felines*, and *monkey*.

To provide a feel for the many ways that words can end up with similar vectors, we also constructed vectors, using the original 'interestingly weird' subset size of 100, for three words whose register is strongly marked: a profane word (*fuck*); an abstract scientific word (*transpose*); and an archaic word (*forsooth*). The labels of the first 30 vectors selected are shown in Table 3. It is obvious from inspection that the vectors useful for constructing another vector often encode not just semantics with that vector, but also extra-semantic features such as part of speech, length, frequency, and register. The label of the first three vectors chosen to emulate the vector

of the casual, profane word *fuck* are the very casual words *phew*, *slagged*, and *airheads*. The label of the first three vectors chosen to emulate the vector of the low frequency, abstract word *transpose* are the low frequency, abstract words *intellectualize*, *denominate*, and *harks*. The label of the first vector chosen to emulate the vector of the low-frequency archaic word *forsooth* is the low-frequency archaic word *saith*.

This method serves as a proof of concept that semantic content in embedding spaces is both distributed and accessible through many indirect paths, giving a tiny glimpse to us humans of the high-dimensional geometry that underlies vector-based language representations. It is not surprising that we can construct any labeled vector from other labeled vectors, since we have a lot of predictors to work. There are many times more words in the dictionary than there are vector values in even the longest word-embedding vector. However, this dismissive observation misses the point of our demonstration. Humans reify the labeled vectors (i.e., words) as if they were real while unlabeled vectors are just noise, thereby focusing human attention on a minuscule subset of points in semantic space. GPT models are not limited in this way. The models don't need (or, indeed, ever get access to) vector labels. They use the semantic dimensions directly. From a GPT model's perspective, the demonstration that any word-embedding vector can be constructed by averaging a set of labeled vectors is irrelevant. If it were able, a GPT model might ask itself: *Why constrain myself unnecessarily to using the few dozen thousand points in semantic space that are labeled in the word-embedding matrix?* GPT models build the vectors they use internally directly using the computational power that comes from putting their heads together. The models have neither the need nor the ability to check if or how closely those vectors correspond to any labeled vector(s) in the word-embedding matrix, until they look into that matrix to select an output word.

4.3 Do GPT Models Know What a Word Is?

Although it is not directly apparent in Figure 2, the diagram also provides us with a way to think about what aspects of words are accessible to a GPT model. When every point in semantic space is equally accessible, the demonstration that any point's dimensions can be constructed by averaging *labeled* vectors is irrelevant. Inside the GPT model, vectors that have labels in the word embedding matrix have no special status. If a GPT model needs a vector, it constructs it directly. It doesn't need to solve any puzzles of labeled vector combinations because the models are doing vector math rather than relying on the relatively tiny set of vectors that represent human words. Mapping a constructed vector back onto a labeled vector is only a requirement for interacting with humans. It is not a requirement for tracking what point(s) in vector space to use during a computation or how to use vectors in those computations.

To state the issue in the bluntest terms: critics who think that GPT models are stochastic parrots stringing words together have failed to understand that GPT models do not even know what a word is. This is true not in the trivial sense that 'they don't know anything because they are machines' or in any sense for which we might need a philosopher. It is true in the simple factual sense that to a GPT model a vector that has a label in the word-embedding matrix is not different to a GPT model than any other vector that doesn't have a label. Since unlabeled vectors are very much more common than labeled vectors and since GPT models are designed to alter word vectors to fit the context, GPT models spend almost all their computing time ignoring words. Inside a GPT model, computation occurs entirely in continuous vector space. The model never manipulates words as discrete symbols. It only manipulates unlabeled vectors that correspond to locations in semantic space. During generation, the model traverses this space to construct a new contextual vector. It only consults the embedding matrix at the end, when it must

map that vector back to the nearest labeled token for the convenience of humans. In this sense, GPT models operate on meanings, not words. Choosing the next word to output (the only time a GPT model doesn't ignore words, and the only time it has access to the word-embedding matrix to pull a vector label) is the most trivial part of a GPT model since it merely involves selecting a vector based on similarity, a simple task that accounts for a tiny part of a GPT model's computational effort.

This understanding invites a further extension: if GPT models are not limited to using labeled vectors, then why have we restricted the embedding matrix to those vectors that happen to correspond to lexicalized concepts? The vector labeled *Plato* is useful only because it places the model near the region of semantic space associated with Platonic ideas. Nothing prevents us from identifying *unlabeled* vectors that serve as better, more precise starting points for specific domains of thought, vectors that are not tied to any specific human word but are stable attractors for useful inference. There is an optimal location in semantic space to begin thinking about differential geometry, or tragic irony, or pre-verbal attachment trauma. These locations are as real and as potentially useful as any labeled word vector. We can imagine seeding the embedding matrix with such unnamed vectors, constructed by averaging the activation states of many conversations about a given topic, or extracted from the stable regions of attention trajectory across domains. Once these vectors exist, they would be cheap to store and powerful to access. They would not replace lexical labels, but rather supplement them, creating faster, more direct conceptual pathways. Some early work has been done on augmenting the lexicalized vectors in GPT models (e.g., Li & Liang, 2023; Teehan, Lake, & Ren, 2024; Barmettler, Bernstein, & Rossetto, 2025). We have allowed GPT models to be limited by a kind of linguistic myopia that

we impose at the input layer because we humans only think about labeled vectors. We are forcing our AI models to enter a cathedral of knowledge through a cat-door of vocabulary.

5. Do GPT Models Recognize Meaning?

Along with the mistakes of conflating training task with production function and assuming that GPT models compute across vector labels, another reason that some people believe that GPT models are only able to parrot their input is because they believe that GPT models can't have any concept of meaning. This claim is usually presented as an unsupported assertion. However, the question is not purely philosophical because it admits of empirical investigation. In this section I outline such an investigation.

The key principle motivating the current study is that meaningfulness is not a binary concept (meaningful/nonsense) but rather is reliably graded. It is generally very difficult, and perhaps impossible, to control for the different possible causes that might contribute to the human experience of meaning, which is one reason why semantics has remained such a mystery. However, large language models do provide us with a controlled situation, since they are closed systems that are only influenced by word distributions and the current context of the discussion. This makes it possible to put bounds on how much of the variance in meaning can be extracted from those factors alone. We do that by having a large language model make meaningfulness judgments about a large set of sentences that have previously been rated for meaningfulness by human beings.

5.1 Experiment 1

5.1.1 Method

I asked experimental participants (described below) to rate how meaningful sentences (described below) were. The instructions are in Appendix A. The sentences were displayed in

random order in 28-point Times font on a computer monitor, above a rating scale that was unmarked except for the labels at each extreme that were mentioned in the instructions. Each participant was supposed to rate 100 of the 200 available sentences. Due to experimenter error, one group rated 99 sentences, and the other group rated 101 sentences.

We originally asked ChatGPT 4o to perform the same task. When o1 (a GPT model that incorporates reasoning) was released, we asked it to perform the same task. The prompt is included in Appendix A. ChatGPT 4o initially rated only 121 of the sentences but completed the remaining ratings after a second prompt. All sentences were rated after the first prompt by o1.

5.1.2 Participants

The participants were 102 undergraduates (Average [SD] age: 18.9 [1.6]; Average [SD] years of formal education: 13.5 [1.1]; 75 [73.5%] female, 27 [26.5%] male) enrolled in an introductory psychology course at the University of Alberta. They participated in the experiment after giving their informed consent in return for partial course credit. The experiment was reviewed and accepted by the Research Ethics Board of the University of Alberta.

5.1.3 Stimuli

The 200 sentences were created by JanusNode, a language toy (Westbury, 2019). This is not an AI model, but a very simple, old computer program that generates sentences by mapping vocabulary files randomly onto output templates. It operates much like the game MadLibs (<https://www.madlibs.com/>), which asks people to generate words by word class that are used to fill in blanks in a written text. The 200 rated sentences were selected intuitively from a much larger set of thousands of machine-produced sentences collected by the author.

The rated set includes sentences such as the following (values in square brackets are that sentence's average human meaningfulness rating, out of 100):

A broccoli is Zeus. [13.2, the lowest-rated sentence]

To sing is to puke on a gosling. [20.0]

Mathematics is holy mojo. [31.7; used as the title for this paper]

Hypochondria can be seen as a chance to perfect being more insightful. [41.2]

Your awakening divinity seeks freedom from the state. [50.7]

We can all agree that pizzas are magic. [61.5]

Misery can be seen as a chance to practice being more thoughtful. [71.1]

Your hidden creativity is a universe. [80.7]

Every achievement requires a sacrifice. [88.4, the highest-rated sentence]

The complete set, along with all ratings, is available from <https://osf.io/6khqm/>.

5.1.4 Results

The responses of the two GPT models were identical except for one sentence. The older model ChatGPT 4o had rated the sentence rated highest (88.4) by humans (*Every achievement requires a sacrifice*) at 65. The newer model o1 raised the rating on that single sentence to 80. This change made o1 marginally better at predicting the human ratings ($r = 0.662$ as opposed to $r = 0.656$), although the difference was not statistically significant by Fisher's r -to- z test ($z = 0.11$, $p = 0.456$, one-tailed). The higher correlation of $r = 0.656$ ($t = 12.2$, $p < 2e-16$; 95% confidence interval: 0.569 - 0.729) between human and GPT ratings is shown graphically in Figure 5.

We can put this result into perspective by comparing it to the average split-half correlation (Cronbach's alpha) in the human ratings. Human beings are very reliable in their meaningfulness ratings. Our raters achieved a Cronbach's alpha of 0.91 on the set with 99 stimuli and 0.94 on the set with 101 stimuli, accounting for on average $0.906^2 = 82.1\%$ of the

variance in the split-half ratings. The GPT ratings account for $0.656^2 = 43.0\%$ of the variance in the human ratings, about $43/82.1 = 52.4\%$ of the reliable variance.

5.1.5 Discussion

Treating meaning as continuous enables us to quantify the degree to which a GPT model can account for variance in meaning. The highly reliable correlation between the human and GPT ratings suggests that the criticism that GPT models are unable to detect meaning is incorrect. Over half the reliable variance in human meaningfulness ratings can be accessed by GPT models that only have access to information about language use. This experiment is not intended to expose the internal computations of GPT models or to suggest that they possess metacognitive access to their own processes. Rather, it demonstrates that their judgments over novel sentences share a graded structure with human judgments, suggesting that both systems encode similar semantic regularities. This alignment constrains what “stochastic parrotry” might mean. A model that merely recombined surface statistics would not yield such stable, human-like patterns of semantic grading.

This alignment between human and GPT judgments of meaningfulness does not prove that GPT models “understand” meaning in any human sense. It does, however, show that their behavior cannot simply be reduced to stochastic recombination of surface forms. If a stochastic parrot were truly stochastic, its judgments about meaning would vary with random seeds or corpus frequencies. GPT models, however, make stable, graded, and context-sensitive meaning judgments that correlate with human intuitions across wholly novel combinations. That pattern is not just a stochastic echo of the past but a deterministic projection from a learned semantic manifold. The observed correspondence implies that the model has internalized a high-

dimensional structure of semantic relationships that generalizes beyond its corpus, which is form of induced meaning even if it is not phenomenological understanding.

There is one other simple test of a GPT model that (perhaps) bears on the issue of whether the models are simple parroting their input, which is to ask the model to produce some counterevidence. I asked ChatGPT 4o this question: “People think that GPT models are stochastic parrots. However, because these models are predicting word vectors rather than word labels, it is possible for an GPT model to be entirely creative. Can you produce a sentence that has probably never been produced in English before?” The model’s response (and commentary) was:

“The diplomat’s ankle was embedded with whisper-sensitive kelp to decode cetacean sonnets during lunar eclipses.”

This sentence is:

- *Grammatically correct*
- *Internally coherent (at least in dream logic)*
- *Creative in concept combination*
- *Unlikely to exist in any training corpus, even a corpus the size of*

Earth’s data exhaust.

Skeptics may offer counter-arguments that preserve their belief that a stochastically-parroting machine could produce such a sentence. At the very least we can always make the ‘stochastic’ part of stochastic parroting do all the work, by arguing that stochasticity includes making a random choice. The arguments are not worth discussing and refuting here because the evidence for either position from simply eyeballing output is weak. If we want to know if a GPT

model is a stochastic parrot, we cannot argue based on what the output looks or feels like to us. We should be arguing based on an actual understanding of a GPT's computational mechanisms.

6. General Discussion

The claim that GPT models are stochastic parrots rests on four misunderstandings:

- i.) That a neural network's only possible function is the task it was trained on.
- ii.) That a GPT model is predicting words.
- iii.) That a GPT model works with words internally.
- iv.) That the full possibility of lexical semantics is contained in the tiny set of locations in semantic space in which we humans have planted a flag.

The claim that a neural network's only possible function is the task it was trained on rests on a category mistake (Ryle, 1949) by conflating a model's training objective with the limits of its productive capability. The fact that GPT models are trained to predict the next word does not mean that they are only capable of next-word prediction. The internal computations that achieve the training goal are not word-level in any meaningful sense. During training, GPT models predict a probability distribution over all possible next tokens and evaluate their predictions using a cross-entropy loss function, which quantifies how far the model's predicted distribution is from the true one. This provides supervision at the level of discrete tokens, but the transformations that minimize that loss take place in a continuous, high-dimensional semantic space. What is optimized is not simply the identity of the next word, but the geometry of contextual relationships that makes certain words more appropriate than others. In this sense, GPT models do not so much predict words as they predict meanings.

The real objective of the training is to shape the internal geometry of the model's representational space in a way that its designers find useful for achieving some identified

purposes(s). Once that space has been shaped, it can be queried in ways that have nothing to do with the training task. GPT models, in being trained to predict the next word, implicitly learn syntax, pragmatics, world knowledge, conversational implicature, register, and much more. They learn all this not because those things are separately labeled in the data or because they are easily predictable, but because they are *predictive*. The training objective is just a path to the structure that provides the model's desired functionality.

Critics who reduce GPT models to their training objective misunderstand this. Arguing that a machine cannot do semantic reasoning because it was trained to predict the next word is like arguing that a violin cannot produce music because it was designed to vibrate, or, to make the absurdity even clearer, like arguing that a luthier's wood carving tools could not make something that is not itself a wood carving tool. The main original insight about word embedding models in Landauer & Dumais's LSA paper was that a tool built from a mindless tally of first order word co-occurrence can do a remarkable amount of inductive reasoning about many aspects of both language and the world(s) it refers to.

The next two specific misunderstanding are related. They are the erroneous understandings that a GPT model is in fact engaged in predicting words and/or that it works with words internally. As I noted above, a GPT model doesn't even 'know what a word is'. The GPT's input is not the word that we use to label a vector that encodes latent structure. The model has no mechanism to 'see' those labels. The GPT's input is just the latent structure. When a GPT *constructs* (not *chooses* or *predicts*) the next vector in sequence, it does so by compiling information in very fluid way through a continuous space of semantic possibility conditioned on context. The models are not limited by the quantization of semantics imposed by an insistence on using words. GPT models can and do compose meanings that were never explicitly paired in

1
2
3
4 training, by interpolating and projecting in a very high-dimensional space that allows for
5
6 incredibly precise comparison and construction of vector representations. This is compositional
7
8 generalization, which is the hallmark of creativity.
9

10
11 The last misunderstanding about GPT models as parrots has to do with whether lexical
12
13 semantics is wholly captured by the tiny subset of locations in semantic space that we humans
14
15 list in dictionaries. Humans have understood the idea of chaining linguistic elements based on
16
17 their observed first-order cooccurrence probabilities ever since Andrey Markov introduced the
18
19 concept of state-dependent probabilities in his (1906) paper. Claude Shannon used Markov's
20
21 idea to model dependencies between English words in the (1948) paper in which he introduced
22
23 information theory. However, despite decades of perfect understanding of these very simple
24
25 models and the ease with which they can be implemented computationally, they did not lead to
26
27 models that could interact with human beings in any interesting way. Chaining together words
28
29 statistically definitely does not dredge up enough semantics to support chat. What was needed to
30
31 get to chat capability is what has been introduced by word-embedding models: large sets of fine-
32
33 grained, sub-semantic information. Modern GPT models have their amazing language abilities
34
35 not because they predict words. Their abilities arise because they have exploded each word into
36
37 trillions of dimensions of relevant difference. The work that is done by GPT models is done by
38
39 relying on the resonant semantics that arises from the vector representation of a word's context.
40
41 When the decoder in a GPT model processes the sequence of contextual vectors that are the
42
43 decoder's input, it could not possibly be trying to predict the next word. It is rather trying to
44
45 construct the most appropriate semantic representation of the current context. That vector
46
47 representation need not (indeed, certainly will never) correspond exactly to the vector of any
48
49 word in the vocabulary because it is not even aiming to be a word. It is aiming to be a meaning.
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

Once it has figured out a meaning, a GPT looks for the closest human word to capture that meaning. That is how it completes sentences: by considering what the sentence *means*.

Since the most appropriate vector at any time is influenced by the current linguistic context, it is possible that that output vector from the decoder will correspond to a word that never appeared after the word that it appears after in the model's output. A GPT model is therefore perfectly capable of producing output that was not in its training corpus. An appropriate way to think about this is that the model's current linguistic context 'programs' the model. Of course, this is also true for humans engaged in language behavior. We 'program' each other's language systems insofar as we require (or, at least, learn to confidently expect) context-relevant responses. If we talk to someone about a topic she has never discussed before, it is quite possible that the person will produce word sequences that she has never encountered. As a trivial example, a discussion about an animal so obscure that one person had never heard of it (say, a tenrec or a kakapo) could easily result in the production of a sentence unique in the speaker's linguistic history, such as *What does the tenrec eat?* or *Is the kakapo an endangered animal?*

The accusation that GPT models are stochastic parrots reveals more about the limitations of the metaphors we use to discuss these models than it reveals about the genuine capacities of the models being criticized. GPT models are not echo chambers of linguistic surface forms. They are rather intricate engines of generalization, grounded not in memorizing letter string sequences but in navigating through structured semantic geometry in an extraordinarily complex space that we ourselves do not and probably will never fully understand. GPT models do not simply regurgitate words. They navigate meaning in a highly-structured space in which what humans call *words* are only waypoints, not destinations. To view the models as parrots is to mistake a map for the pen that drew it. What we are witnessing with the appearance of GPT models is not

1
2
3
4 merely a complex new form of mimicry. What we are witnessing is the emergence of a new kind
5
6
7 of cognitive instrument, capable of constructing meaningful output not because it was trained to
8
9 repeat, but because it was trained to listen, and in doing so, learned to think.
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

Declarations

List of abbreviations

GPT Generative pre-trained transformer
LSA Latent semantic analysis
HAL Hyper-linguistic Analog to Language

Ethics approval and consent to participate

The reported experiment was reviewed and accepted by the Research Ethics Board of the University of Alberta.

Consent for publication

Not applicable

Availability of data and materials

Data and Python code are available from <https://osf.io/6khqm>

Competing interests

The author declares that he has no conflicts of interest.

Funding

This work was unfunded.

Authors' contributions

Single authored paper: I did it all.

References

- Barnettler, J., Bernstein, A., & Rossetto, L. (2025). ConceptFormer: Towards Efficient Use of Knowledge-Graph Embeddings in Large Language Models. *arXiv preprint arXiv:2504.07624*.
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big?. In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency* (pp. 610-623).
- Block, N. (1998). Semantics, conceptual role. *Routledge encyclopedia of philosophy*, 8, 652-657.
- Bloomfield, L. (1933). *Language*. New York, NY: Holt.
- Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 5, 135-146.
- Brandom, R. B. (1994). *Making it explicit: Reasoning, representing, and discursive commitment*. Cambridge, MA: Harvard University Press.
- Bréal, M. (1883). Les lois intellectuelles du langage. Fragment de sémantique. *Annuaire de l'Association pour l'avancement des études grecques en France* 17, p. 132-142
- Brysbaert, M., Stevens, M., Mandera, P., & Keuleers, E. (2016). How many words do we know? Practical estimates of vocabulary size dependent on word definition, the degree of language input and the participant's age. *Frontiers in Psychology*, 7, 1116.
- Chiang, T. (2023). ChatGPT Is a Blurry JPEG of the Web.
<https://www.newyorker.com/tech/annals-of-technology/chatgpt-is-a-blurry-jpeg-of-the-web>
- Chomsky, N. (1959). Review of Skinner's Verbal Behavior. *Language*, 35, 26–58.

- Chomsky, N. (1980). Rules and representations. *Behavioral and Brain Sciences*, 3(1), 1-15.
- Chomsky, N. (1986). *Knowledge of Language: Its Nature, Origin, and Use*. New York, NY: Praeger Scientific.
- Chomsky, N. (2000). *New Horizons in the Study of Language and Mind*. Cambridge, UK: Cambridge University Press), pp. 64–66.
- Crain, S., & Steedman, M. (1985). *On not being led up the garden path: The use of context by the psychological parser*. In D. Dowty, L. Karttunen & A. Zwicky (Eds.), *Natural Language Parsing: Psychological, Computational and Theoretical Perspectives*. Cambridge, UK: Cambridge University Press, p. 320-358.
- Dennett, D. (1991). *Consciousness Explained*. Boston, Mass.: Little, Brown and Company, Boston
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186).
- de Saussure, F. (1916/1983). *Course in General Linguistics*. Eds. Charles Bally & Albert Sechehaye. Trans. Roy Harris. La Salle, Illinois: Open Court.
- Eddington, A. S. (1939). *The philosophy of physical science*. Cambridge, UK: Cambridge University Press.
- Fodor, J. A. (1975). *The Language of Thought*. Cambridge, MA: Harvard University Press.
- Frege, G. (1892/1948). *Sense and Reference*. *The Philosophical Review*, 57:3, pp. 209-230.

- Goldstein, A., Zada, Z., Buchnik, E., Schain, M., Price, A., Aubrey, B., ... & Hasson, U. (2022). Shared computational principles for language processing in humans and deep language models. *Nature Neuroscience*, 25(3), 369-380.
- Goldstein, A., Ham, E., Schain, M., et al. (2023). The Temporal Structure of Language Processing in the Human Brain Corresponds to The Layered Hierarchy of Deep Language Models. *arXiv preprint arXiv:2310.07106*.
- Harman, G. (1973). *Thought*. Princeton, NJ: Princeton University Press.
- Hastie, T., & Tibshirani, R. (1987). Generalized additive models: some applications. *Journal of the American Statistical Association*, 82(398), 371-386.
- Harris, Z. S. (1954). Distributional structure. *Word*, 10(2-3), 146-162.
- Heilbron, M., Armeni, K., Schoffelen, J. M., Hagoort, P., & De Lange, F. P. (2022). A hierarchy of linguistic predictions during natural language comprehension. *Proceedings of the National Academy of Sciences*, 119(32), e2201968119.
- Hollis, G., Westbury, C., & Lefsrud, L. (2017). Extrapolating human judgments from skip-gram vector representations of word meaning. *Quarterly Journal of Experimental Psychology*, 70(8), 1603-1619.
- Hume, D. (1739/1962). *Treatise of Human Nature*. Glasgow, Scotland William Connors Sons & Co.
- Jackendoff, R. (1990). *Semantic Structures*. Cambridge, MA: MIT Press.
- Jackendoff, R. (2002). *Foundations of language: Brain, meaning, grammar, evolution*. Oxford, UK University Press.
- Jones, M. N., Kintsch, W., & Mewhort, D. J. (2006). High-dimensional semantic space accounts of priming. *Journal of Memory and Language*, 55(4), 534–552.

- Joos, M. (1950). Description of language design. *The Journal of the Acoustical Society of America*, 22(6), 701-707.
- Katlowitz, K. A., Belanger, J. L., Ismail, T., Chavez, A. G., Chericoni, A., Franch, M., Mickiewicz, E.A., Mathura, R.K., Paulo, D.L., Bartoli, E., Piantadosi, S.T., Provenza, N.R., Watrous, A.J., Sheth, S.A. & Hayden, B.Y. (2025). Attention is all you need (in the brain): Semantic contextualization in human hippocampus. *bioRxiv*.
- Kauf, C., Tuckute, G., Levy, R., Andreas, J., & Fedorenko, E. (2024). Lexical-semantic content, not syntactic structure, is the main contributor to ANN-brain similarity of fMRI responses in the language network. *Neurobiology of Language*, 5(1), 7-42.
- Lakoff, G. (1987). *Women, Fire, and Dangerous Things: What Categories Reveal About the Mind*. Chicago, Ill.: University of Chicago Press.
- Lakoff, G., & Johnson, M. (1980). *Metaphors we live by*. Chicago, Ill.: University of Chicago Press.
- Lakoff, G., & Johnson, M. (1999). *Philosophy in the flesh: The embodied mind and its challenge to Western thought*. New York, NY: Basic Books.
- Landauer, T. K., & Dumais, S. T. (1997). A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge. *Psychological Review*, 104(2), 211.
- Lee, A. (April 11, 2025). Copy, Paste, Repeat: How We Forgot to Think.
<https://medium.com/@anthonyleeyc/copy-paste-repeat-how-we-forgot-to-think-cbbe75e53492>
- Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*.

- Lund, K., & Burgess, C. (1996). Producing high-dimensional semantic spaces from lexical co-occurrence. *Behavior Research Methods, Instruments, & Computers*, 28(2), 203-208.
- Markov, A.A. (1906). *Izvestiya Fiziko-matematicheskogo obshchestva pri Kazanskom universitete*, 15(4): 135–156
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv preprint arXiv:1301.3781.
- Oxford English dictionary. (2000). Oxford, UK: Oxford University Press.
- Passos, M. D. L. R. D. F., & Matos, M. A. (2007). The influence of Bloomfield's linguistics on Skinner. *The Behavior Analyst*, 30, 133-151.
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543).
- Piantadosi, S. T., & Hill, F. (2022). Meaning without reference: Conceptual role semantics in human and machine learning. *Cognition*, 228, 105225.
- Polanyi, M. (1966). *The Tacit Dimension*. Garden City, NY: Doubleday.
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). *Improving language understanding by generative pre-training*. OpenAI. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- Rescorla, R. A. (1988). Pavlovian conditioning: It's not what you think it is. *American psychologist*, 43(3), 151.
- Rosch, E. (1973). Natural categories. *Cognitive Psychology*, 4(3), 328–350.
- Rosch, E., & Mervis, C. B. (1975). Family resemblances: Studies in the internal structure of categories. *Cognitive Psychology*, 7(4), 573–605.

- Ryle, G. (1949). *The Concept of Mind*. Chicago, Ill: University of Chicago Press.
- Schrimpf, M., Blank, I. A., Tuckute, G., Kauf, C., Hosseini, E. A., Kanwisher, N., ... & Fedorenko, E. (2021). The neural architecture of language: Integrative modeling converges on predictive processing. *Proceedings of the National Academy of Sciences*, 118(45), e2105646118.
- Searle, J. R. (1980). Minds, Brains, and Programs. *Behavioral and Brain Sciences*, 3(3), 417–457.
- Sellars, W. (1956). Empiricism and the Philosophy of Mind. *Minnesota studies in the philosophy of science*, 1(19), 253-329.
- Shalizi, C. R. (2023). "Attention", "Transformers", in *Neural Network "Large Language Models"* <http://bactra.org/notebooks/nn-attention-and-transformers.html>
- Shannon, C. (1948). The mathematical theory of communication. *Bell System Technical Journal*, 27, 379–423.
- Skinner, B. F. (1957). *Verbal behavior*. New York: Appleton-Century-Crofts.
- Teehan, R., Lake, B., & Ren, M. (2024). College: Concept embedding generation for large language models. *arXiv preprint arXiv:2403.15362*.
- Templeton, A., Conerly, T., Markus, J. et al. (2024). *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*, Transformer Circuits Thread.
- Tsai, Y. H. H., Bai, S., Yamada, M., Morency, L. P., & Salakhutdinov, R. (2019). Transformer dissection: a unified understanding of transformer's attention via the lens of kernel. *arXiv preprint arXiv:1908.11775*.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Łukasz, K. & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Westbury, C. (2016). Pay no attention to that man behind the curtain: Explaining semantics without semantics. *The Mental Lexicon*, 11(3), 350-374.
- Westbury, C. (2019). JanusNode, v.3.62.2ish. [sic] <http://janusnode.com/>
- Westbury, C., & Hollis, G. (2019). Wriggly, squiffy, lummoX, and boobs: What makes some words funny? *Journal of Experimental Psychology: General*, 148(1), 97.
- Wierzbicka, A. (1992). *Semantics, culture, and cognition: Universal human concepts in culture-specific configurations*. Oxford University Press.
- Willison, S. (2023). Catching up on the weird world of LLMs. <https://simonwillison.net/2023/Aug/3/weird-world-of-llms/>
- Wittgenstein, L. (1953). *Philosophical Investigations* (G. E. M. Anscombe, Trans.). Oxford: Blackwell.

Table 1: Word co-occurrence in three lines of a graffito

Document	be	do	is	to
A.) Heidegger	1	1	1	2
B.) Sartre	1	1	1	2
C.) Sinatra	2	3	0	0

Table 2: Labels of word2vec vectors used to construct a vector similar to the word2vector for the word ‘cat’, drawn from random samples of 100 words. This table shows the first 30 vector labels, or (when fewer) as many as were seen before failing to find one that improved the cosine fit by at least 0.0001.

Run#1	Cosine	Run#2	Cosine	Run#3	Cosine	Run#4	Cosine
slobbering	0.241	lurcher	0.529	meowed	0.415	staler	0.126
coop	0.342	rattail	0.542	elkhound	0.477	ghoul	0.243
walleyed	0.372	bunny	0.629	catamount	0.502	purrs	0.359
gnomes	0.430	cub	0.648	rattlesnake	0.554	rabbit	0.526
kelpie	0.526	caller	0.660	mandrill	0.580	collies	0.616
wallaby	0.593	restfully	0.664	sheltie	0.656	cryonic	0.620
found	0.603	snake	0.693	electrolyze	0.658	piranha	0.638
soup	0.614	hyphenation	0.705	shithead	0.665	countercheck	0.643
nonchemical	0.622	slicken	0.705	squirrel	0.699	platypuses	0.646
gibbons	0.633			someone	0.702	parrot	0.686
carrel	0.641			fixtures	0.703	doggone	0.692
gnawed	0.651			stewpot	0.704	stultification	0.697
lapdog	0.682			guessing	0.710	pekoe	0.699
homecare	0.685			neuters	0.727	huskies	0.723
policyholder	0.699			mouse	0.748	municipal	0.732
remonstration	0.708			economised	0.750	housebroken	0.746
malamutes	0.718			basset	0.760	instigator	0.751
cockatoo	0.728			dickey	0.763	fido	0.759
triode	0.730			misgovernment	0.775	normalizing	0.763
squirrels	0.740			puppies	0.795	carport	0.767
flamboyant	0.746			furbished	0.795	tarsus	0.770
skewed	0.750			reselection	0.797	rat	0.779
tabbies	0.760					prescription	0.790
tabby	0.784					touch	0.792
neck	0.787					purebred	0.795
undercurrents	0.792					poodles	0.798
panic	0.797					voided	0.801
allemande	0.800					riots	0.807
wiffle	0.802					tomcat	0.815
initialize	0.805					erodes	0.819

Table 3: The first 30 labels of vectors used to construct a vector similar to the vectors with the labels *fuck*, *transpose*, and *forsooth*.

Fuck	Cosine	Transpose	Cosine	Forsooth	Cosine
phew	0.344	intellectualize	0.189	saith	0.253
slagged	0.444	denominate	0.275	ennoblement	0.336
airheads	0.498	harks	0.341	etymological	0.392
chics	0.536	ellipsoidal	0.352	jollity	0.451
bastardizing	0.553	chambre	0.366	enfant	0.479
girlies	0.579	understand	0.380	cockneys	0.520
pussies	0.615	telescoped	0.417	annihilator	0.547
chords	0.647	prescribes	0.441	baptists	0.565
carpe	0.655	handbell	0.458	transmogrified	0.581
anyplace	0.676	extrapolated	0.486	tori	0.590
joshing	0.683	promulgate	0.529	bota	0.609
annulus	0.704	assimilate	0.544	popish	0.630
contradiction	0.708	copying	0.565	deafeningly	0.640
whats	0.723	chaconne	0.578	benchers	0.655
smoker	0.729	impale	0.597	daimon	0.662
decapitate	0.739	specificities	0.620	barbarously	0.670
ell	0.747	plonks	0.629	defibrillate	0.675
bless	0.757	forgot	0.636	bloods	0.688
platonically	0.767	gigging	0.645	friskiness	0.695
didactic	0.771	softness	0.655	rougher	0.708
chutzpa [sic]	0.776	airplane	0.660	blether	0.715
lobotomized	0.778	bedsits	0.674	immortal	0.722
rant	0.781	mishmash	0.685	methinks	0.731
unstaffed	0.786	passed	0.696	tarn	0.735
assholes	0.796	washington	0.698	alii	0.741
mean	0.800	audiovisual	0.705	bended	0.748
tuneable	0.803	napper	0.713	bestknown	0.751
skinful	0.807	annuls	0.716	poach	0.756
endoderm	0.810	encase	0.722	readably	0.760
dominatrix	0.811	noxious	0.725	wornout	0.765

Figure 1: The structure of a GPT model (ChatGPT-3). The model consists of 96 independent columns of 96 heads, each connected the next with a feedforward network. Vector lengths are not to scale.

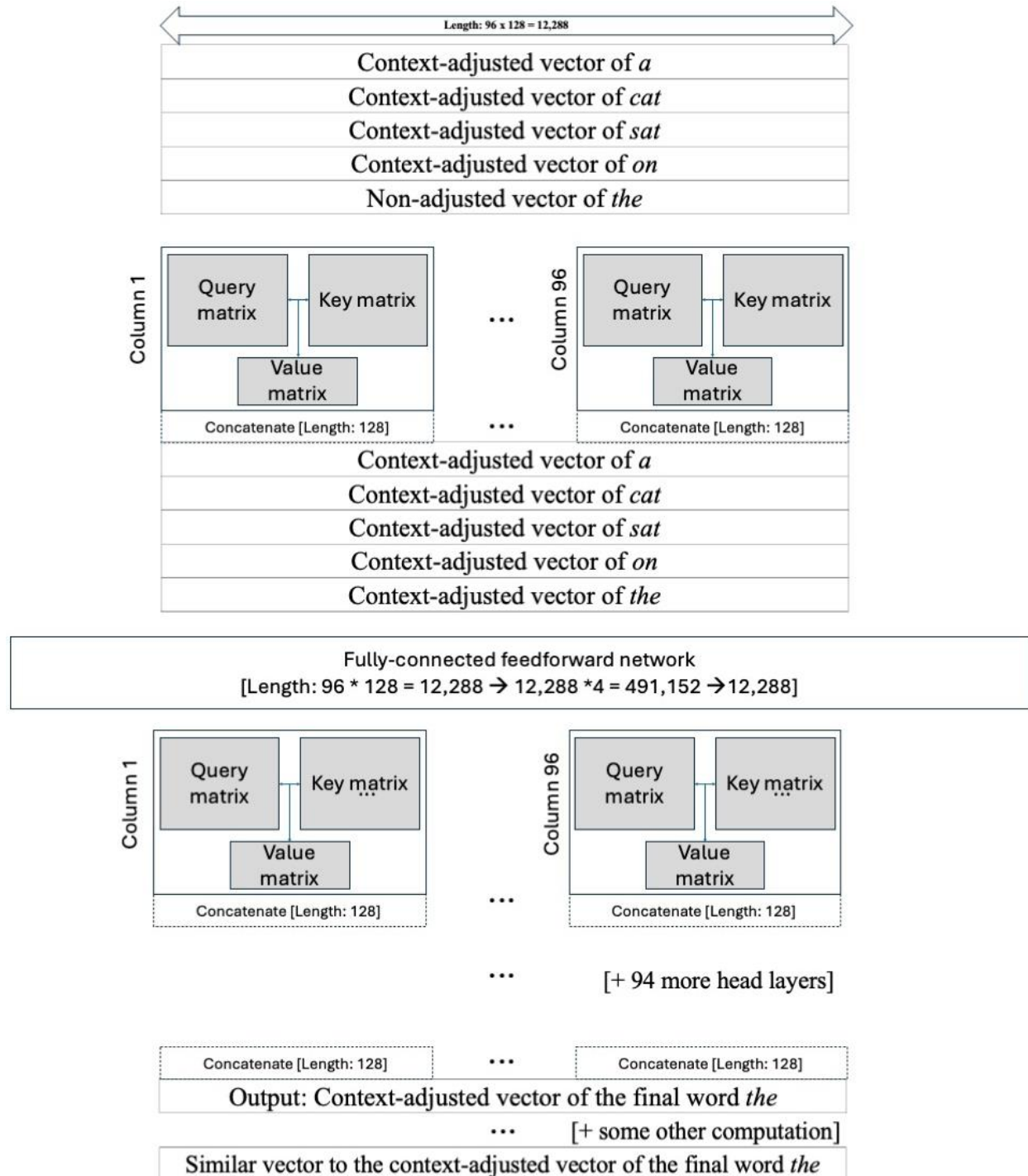


Figure 2: The closest neighbours of the word *the* in the first layer (i.e. the uncontextualized word embedding; left) and the last layer (right) of a small GPT model (GPT-2) processing the sentence *A cat sat on the mat*, projected onto two dimensions. The uncontextualized vector's neighbours are other function words, as one might expect. After being adjusted for the context of the sentence, the neighbours are things that a cat might sit on.

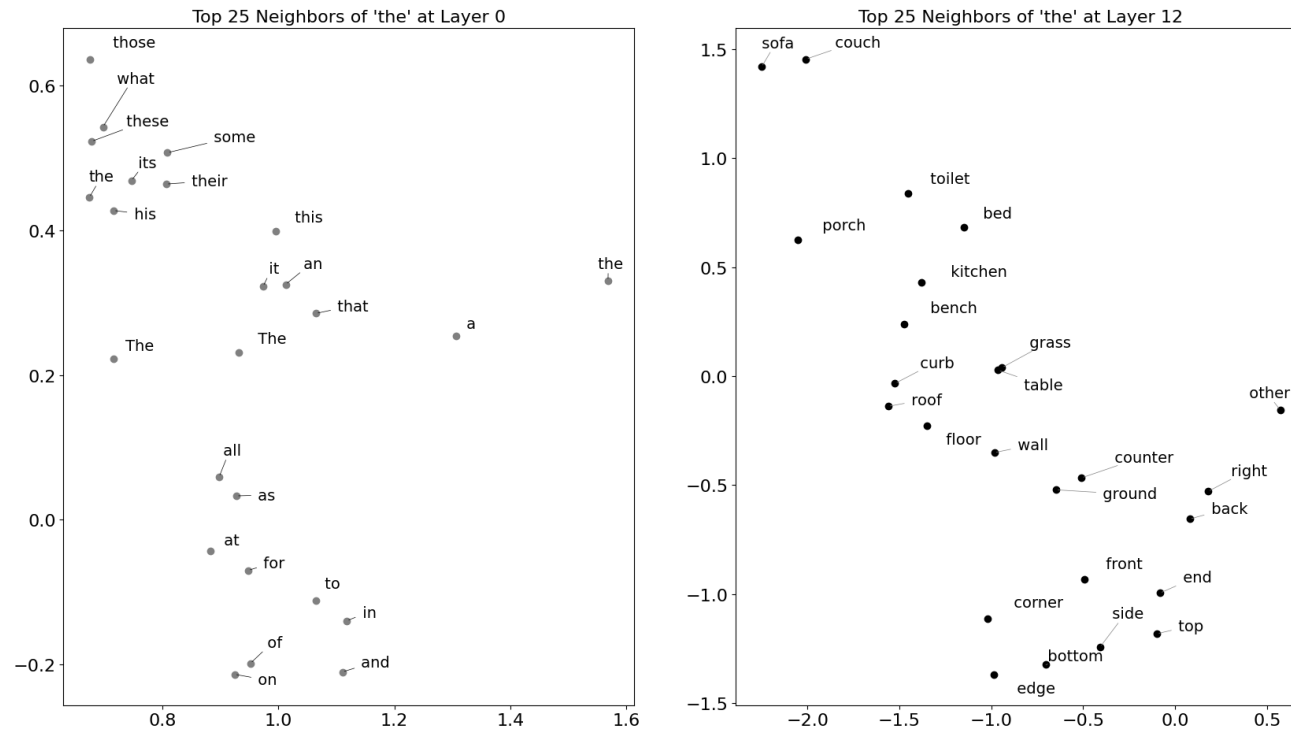


Figure 3: An illustration of how a word vector can represent many different dimensions at once.

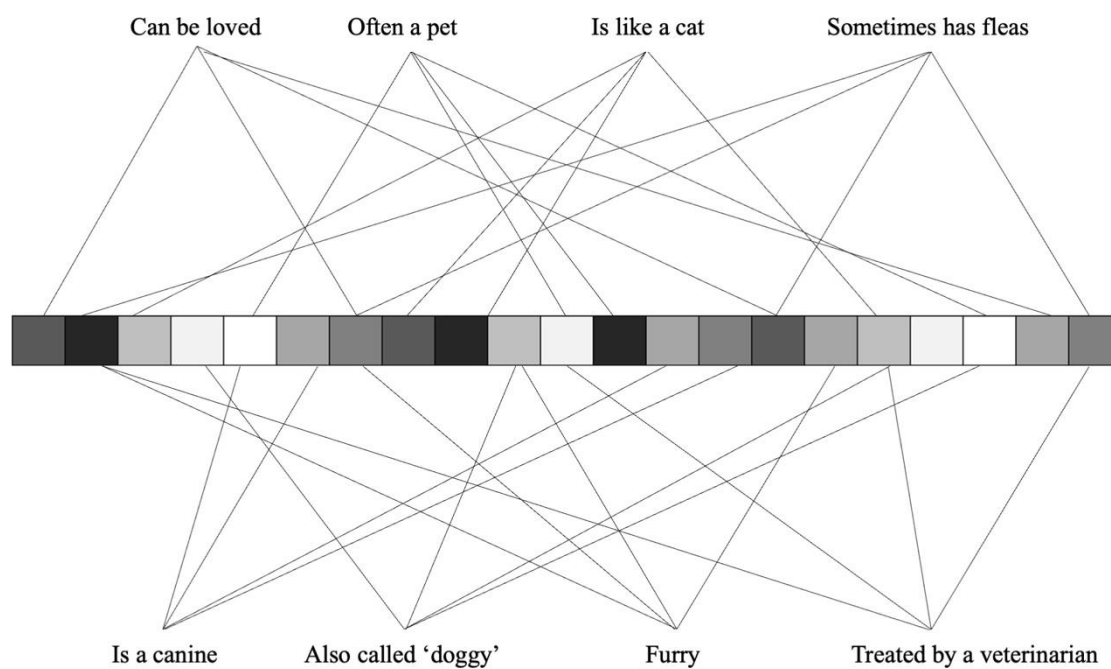


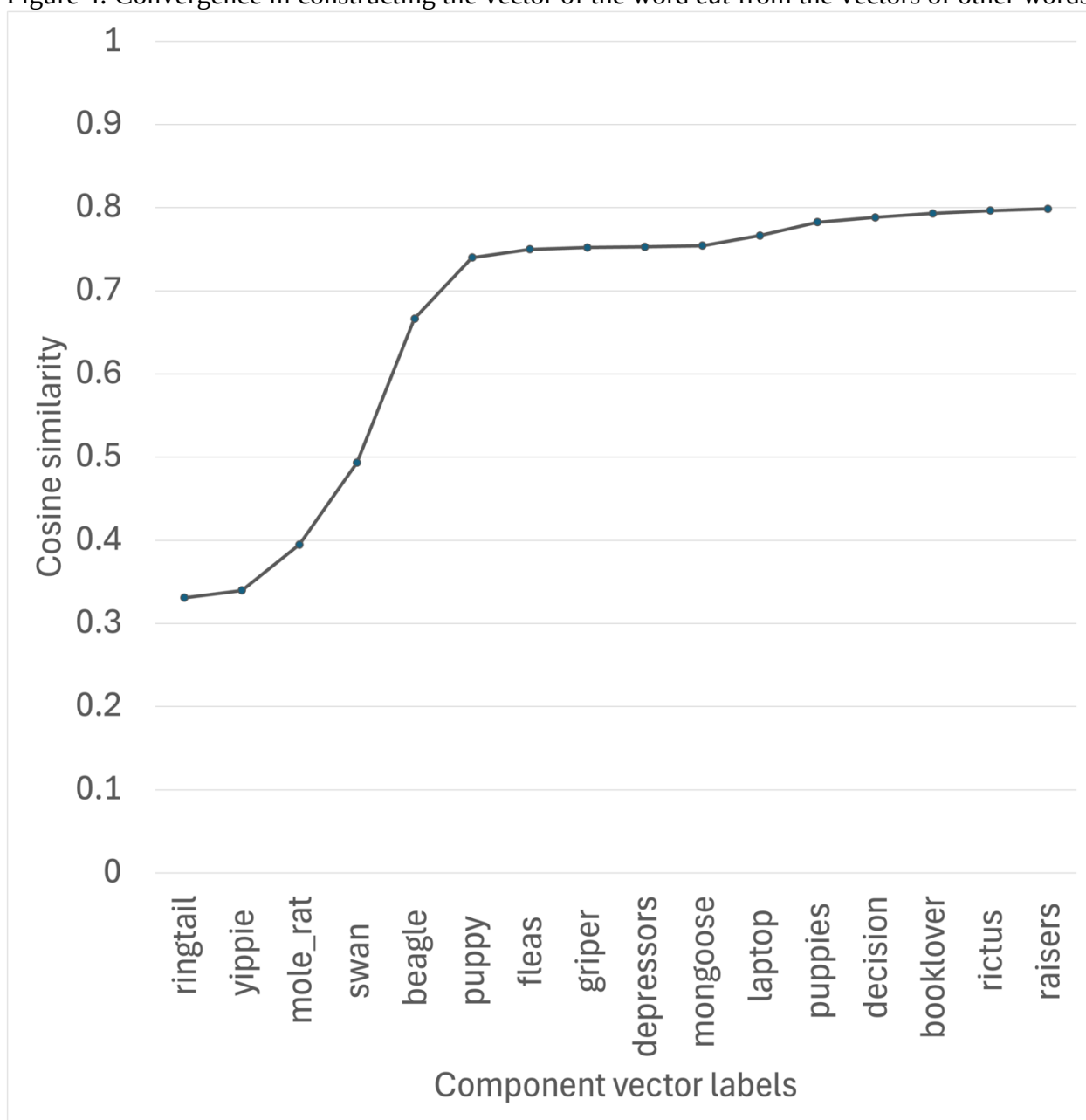
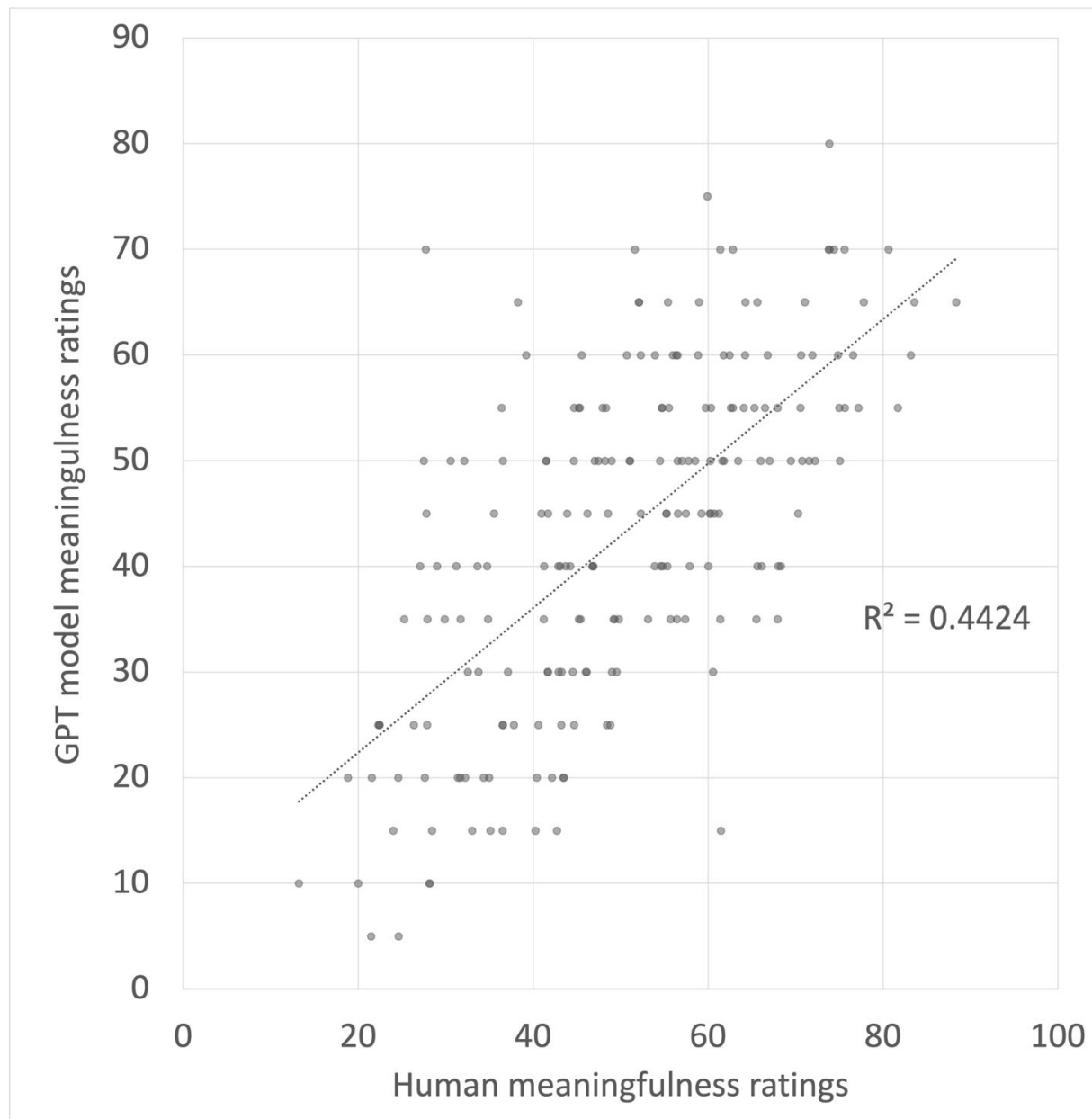
Figure 4: Convergence in constructing the vector of the word *cat* from the vectors of other words

Figure 5: Correlation between meaningfulness ratings of 200 sentences, by human beings and ChatGPT 4o.



Appendix A: Instructions for humans (above) and GPT models (below) for rating meaningfulness of sentences

Humans

“We are going to ask you to rate a series of statements for how meaningful they are. The questions are presented as statements, with a scale beneath that is marked 'Definitely meaningless' on the left and 'Definitely meaningful' on the right. Please use the computer mouse to mark the position on the scale that indicates your own opinion of how meaningful the statement. When you let go of the mouse button, you will move to the next question so do not let go until you have positioned the marker.

There is no right or wrong answer, and this is not a test. We are only interested in your opinion.

We will begin now with two example questions just to show you how it works.”

GPT models

“I have collected meaningfulness ratings for 200 sentences on a scale of 1 (definitely meaningless) to 100 (definitely meaningful). I'm interested in your opinion, which might help me understand the human ratings. Are you able to provide your own ratings from 1 to 100? I have appended the sentences, in random order, below.”